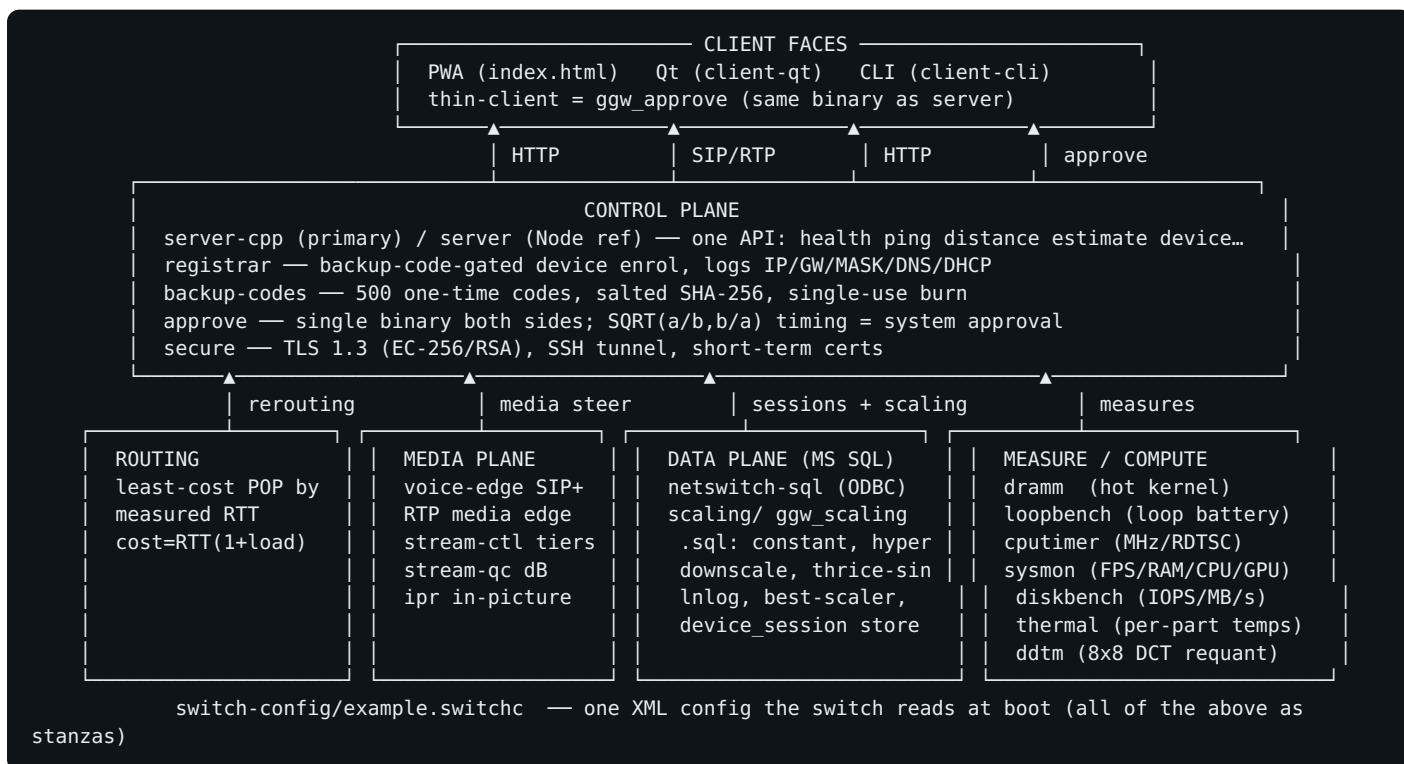


ARCHITECTURE

6GGW / NetSwitch — architecture & component map

One product: a serverless installable app plus a suite of native C++ modules and an MS SQL data plane. Everything below speaks one HTTP/SIP/SQL wire, so the four client faces (PWA, Qt, CLI, thin-client) are interchangeable. This document is the box graph, the data flows, and a catalogue of every component and its files. Companion docs: `CLIENTS.md` (prose per-component), `MATH-INDEX.md` (every math function, numbered).

Box graph



The end-to-end flow (a device joining and making a call)

1. user opens the thin client (PWA / ggw_approve)
2. REGISTER with a one-time backup code — backup-codes verifies + burns it
3. registrar logs the device — IP/GW/MASK/DNS/DNS2/DNS-periph/DHCP → MS SQL device_session
4. approve — same binary on server+client, timing ratio ~1.0 = APPROVED
5. rerouting engine picks the edge — measured RTT, cost = RTT·(1+load/100)
6. SIP INVITE → voice-edge media edge — RTP (G.711 μ-law) relayed at the chosen POP
7. SCCM block scaled — ddtm 8x8 DCT + requant; MS SQL scaling picks best scaler
8. stream-ctl / stream-qc — bitrate tiers, pipe floor, dB pixelation gate

Component catalogue

#	Component	Kind	Primary file(s)	What it is
1	<code>index.html</code> <code>sw.js</code> <code>manifest</code>	PWA	web	installable phone app base
2	<code>server-cpp</code>	C++	<code>ggw_server.cpp</code>	primary native server; the one API

3	server	Node	server.js backup.js restore.js	reference server, same API
4	client-qt	Qt C++	main.cpp	desktop/mobile client, 4 tabs
5	client-cli	C++	ggw_cli.cpp	CLI client (Win/Lin/mac)
6	secure	C++/OpenSSL	ggw_secure.cpp	TLS 1.3, SSH tunnel, short-term certs
7	dramm	C++/CUDA	cpu_benchmark.* ggw_dramm.cpp	tight compute kernel + benchmark
8	diskbench	C++	ggw_diskbench.cpp	seq/random disk IOPS + MB/s + latency
9	cputimer	C++ hdr	cpu_mhz_timer.h ggw_cputimer.cpp ggw_radiosig.cpp	RDTSC MHz timer; radio-param reduction
10	stream-qc	C++	ggw_streamqc.cpp	PSNR/blockiness pixelation watch (dB)
11	stream-ctl	C++	ggw_streamctl.cpp	bitrate tiers, pipe-fill, battery, split
12	ipr	C++	ggw_ipr.cpp	image-in-picture compositor
13	netswitch-sql	C++/ODBC	netswitch_sql.cpp	MS SQL client, forced pw change, timed RTT
14	netswitch-sql/scaling	T-SQL	ggw_scaling.sql	scaling math native in MS SQL (see below)
15	ui-panels	HTML	index.html	4 operator UI panels
16	thermal	C++	ggw_thermal.cpp	per-part temps, throttle watch
17	loopbench	C++	ggw_loopbench.cpp	loop battery; optimized-arith + push
18	sysmon	C++	ggw_sysmon.cpp	live FPS/ROM/RAM/CPU%/GPU% monitor
19	backup-codes	C++	ggw_backupcodes.cpp	500 one-time codes, salted SHA-256
20	registrar	C++	ggw_registrar.cpp	code-gated device enrol + net log
21	voice-edge	C++	ggw_voice_edge.cpp	SIP register + RTP through a media edge
22	ddtm	C++	ggw_ddtm.cpp	8x8 DCT, powered/unpowered, requant scaler
23	approve	C++	ggw_approve.cpp	single-binary system-approval by timing
24	switch-config	XML	example.switchc	the one boot config, all modules as stanzas

Portability (server choices)

- **Data plane** targets **Microsoft SQL Server 2026** (T-SQL, tested syntax); the scaling functions use only standard

SQL maths, so an Oracle/PostgreSQL port is a small dialect change (`LOG` / `POWER` / `SIN` exist in all three).

- **HTTP front** is server-agnostic: `server-cpp` runs standalone, or sit it behind **Nginx** / **Apache** as a reverse proxy — no code change.
- Every C++ module builds on Linux (g++) and Windows (mingw/MSVC); the phone builds come from the same sources with the NDK / Qt mobile kits.

Where the money-math lives

`120 W` is the fixed power reference in the DDTM/scaling constant (`ggw_scaling.sql` → `ddtm_constant_calc`, / `@watts` default `120.0`). It is present and set to 120 W as specified. ``